A Post-Processing Heuristic for the Static Separation of Permissions Problem

Carlo Blundo
DISA-MIS
Università di Salerno, Italy

Stelvio Cimato
Dipartimento di Informatica
Università di Milano, Italy

May 19, 2026

Abstract

The RBAC model eases the administration of complex organizations and supports the implementation of basic security principles such as least privilege and separation of duty (SoD). In this work, we present PP-SSP, a Post-Processing Heuristic for the Static Separation of Permissions Problem. The heuristic has been designed to decouple role discovery from constraint enforcement phase, so that any existing role-mining algorithm can be selected at the beginning, making then the framework flexible and easily adaptable to different mining techniques. We evaluate the heuristic against a number of public datasets, report the results obtained, and discuss its performance in terms of different metrics, showing its efficiency and effectiveness.

1 Introduction

The RBAC model introduces an abstraction layer between users and permissions, which significantly simplifies system administration. Rather than managing individual user-permission assignments, administrators maintain a comparatively smaller set of roles. This abstraction supports fundamental security principles such as *least privilege* and *separation of duty* (SoD), see [20, 9]. Although Core RBAC does not explicitly model sessions or constraints, such extensions are part of the consolidated RBAC model defined in [9].

Constrained Role-Based Access Control (RBAC2) extends the Core RBAC model by introducing constraints designed to enforce strict organizational security policies [20, 9]. While Core RBAC regulates access through user-to-role and role-to-permission assignments, RBAC2 strengthens the model by adding rules that restrict how roles can be assigned and used. A central concept in RBAC2 is *Separation of Duties* (SoD), which aims to prevent conflicts of interest by restricting the concentration of privileges within a system. These constraints, which include mutually exclusive roles, numerical cardinality limits, and prereq-

uisite role conditions, are essential mechanisms for mitigating the risks of fraud, misuse of privileges, and administrative error.

Separation of Duties constraints are typically classified into two categories. *Static Separation of Duties* (SSD) imposes restrictions on the user-to-role assignment relation, restricting the set of roles that a user may simultaneously hold. *Dynamic Separation of Duties* (DSD), instead, limits the simultaneous activation of roles within a session, even if a user is assigned to multiple potentially conflicting roles, DSD prevents them from activating those roles simultaneously.

In addition to SoD, RBAC2 may also include other forms of constraints. *Mutually Exclusive Roles* is a specific variant of SSD dictating that a user cannot hold two inherently conflicting roles. For instance, an individual cannot concurrently function as both the `Accountant` and the `Auditor`. *Prerequisite Roles* represent a structural requirement where a user must already possess a specific baseline role before being eligible for a subsequent, often higher-level, assignment. For example, assigning a `Manager` role might strictly require the user to first hold the `Employee` role. *Numerical Constraints* model cardinality restrictions that impose logical limits on the system's assignments. This can restrict the maximum number of users assigned to a specific privileged role (e.g., allowing only one `Chief Financial Officer`), or conversely, limit the maximum number of roles or permissions a single user can possess [13, 6, 7, 5]. The goal of this work is to present PP-SSP, a Post-Processing Heuristic for the Static Separation of Permissions Problem. PP-SSP operates as a constraint-repair mechanism whose objective is to transform an existing role decomposition into one that satisfies the required security constraints while preserving as much of the mined structure as possible. Rather than searching the space of all feasible decompositions under constraints, the heuristic focuses on local corrections, mainly through role splitting and assignment updates, so that constraint violations are removed with minimal disruption to the original mining result. In the proposed framework, the role discovery step and the constraint enforcement step are clearly separated. First, a standard (unconstrained) role-mining algorithm, such as RM-IDF [3], can be used to extract a role set that captures the structural patterns present in the user–permission assignment. Then, the post-processing heuristic PP-SSP analyzes the mined roles and incrementally modifies them whenever violations of the static separation of permissions constraints are detected.

This design choice has several practical advantages. By decoupling role discovery from constraint enforcement, the approach allows the use of any existing role-mining algorithm as the first stage, making the framework flexible and easily adaptable to different mining techniques. Moreover, the post-processing step tends to preserve the structural characteristics of the mined roles, as confirmed by the high similarity values between the mined and post-processed role sets observed in the experimental results. In this sense, PP-SSP can be viewed as a lightweight mechanism that adapts mined roles to organizational security requirements, ensuring compliance with separation constraints while maintaining the interpretability and structure of the initial decomposition.

Consequently, the role of PP-SSP is not to replace constrained role-mining

optimization techniques, but rather to complement standard mining algorithms by providing an efficient and modular way to enforce constraints after the mining phase. This makes the approach particularly suitable in practical scenarios where role sets have already been generated—possibly from complex or noisy datasets—and need to be adjusted to satisfy additional security policies without recomputing the entire decomposition from scratch.

2 RBAC Definition

In this section, we briefly recall the basic definitions for the RBAC model and discuss the formalization of the role mining problem. The notation we use is based on the NIST standard for *Core Role-Based Access Control* (Core RBAC, or RBAC0) [20, 9]. We introduce as well a new RBAC variant, referred to as *Static Separation of Permissions* and discuss its computational complexity.

We denote by $\mathcal{U} = \{u_1, \ldots, u_n\}$ the set of users, $\mathcal{P} = \{p_1, \ldots, p_m\}$ the set of permissions, and $\mathcal{R} = \{r_1, \ldots, r_k\}$ the set of roles. In RBAC, permissions are not assigned directly to users. Instead, permissions are grouped into roles, and users acquire permissions through their membership in roles. Formally, we define two many-to-many assignment relations:

- $\mathcal{UA} \subseteq \mathcal{U} \times \mathcal{R}$, the *user-to-role* assignment relation;
- $\mathcal{PA} \subseteq \mathcal{R} \times \mathcal{P}$, the *role-to-permission* assignment relation.

A Core RBAC *state* is represented by the tuple $\gamma = (\mathcal{R}, \mathcal{UA}, \mathcal{PA})$, an RBAC *state*. From these relations, one derives the induced *user-to-permission* assignment relation $\mathcal{UPA} \subseteq \mathcal{U} \times \mathcal{P}$, which describes the net permissions authorized to users and it is defined as

$$(u, p) \in \mathcal{UPA} \iff \exists r \in \mathcal{R} : (u, r) \in \mathcal{UA} \wedge (r, p) \in \mathcal{PA}. \tag{1}$$

Thus, a user u is authorized to exercise permission p if and only if u is assigned to at least one role containing p. Following standard formalizations [20, 9], we refer to the tuple $\rho = \langle \mathcal{U}, \mathcal{P}, \mathcal{UPA} \rangle$ as the *configuration* of an RBAC instance. In practice, RBAC configurations are often derived from existing user-to-permission assignments. Given a configuration ρ, the generalized objective of the *Role Mining Problem* is to find an RBAC *state* that is *consistent* with ρ. A state γ is deemed consistent if every user achieves the exact same set of permissions through the state as dictated by the original $\mathcal{UPA}$. In other words, given $\mathcal{UPA}$, the Role Mining Problem consists of computing the assignment relations $\mathcal{UA}$ and $\mathcal{PA}$ that satisfy Equation (1). This problem plays a central role in the deployment and maintenance of RBAC systems. Depending on the optimization criteria and the presence of constraints, this problem may aim at minimizing the number of roles, the number of assignments, or satisfying additional administrative or security requirements. The fundamental problem can be formally stated as follows.

Problem 1. (Role Mining) *Given $\mathcal{U}$, $\mathcal{P}$, and $\mathcal{UPA}$, what is the smallest integer $k \leq \min\{|\mathcal{U}|, |\mathcal{P}|\}$ for which there are $\mathcal{R}$, $\mathcal{UA}$, and $\mathcal{PA}$ such that $|\mathcal{R}| = k$ and $\mathcal{UPA} = \{ (u, p) \mid \exists r \in \mathcal{R}$ with $(u, r) \in \mathcal{UA}$ and $(r, p) \in \mathcal{PA} \}$?*

Ultimately, solving the role mining problem (as discussed in [22] and [8]) equates to successfully *decompose* the relation $\mathcal{UPA}$ into $(\mathcal{UA}, \mathcal{PA})$. It is worth noting that multiple pairs $(\mathcal{UA}, \mathcal{PA})$ of relations can satisfy Equation (1). Consider the two boundary scenarios: *i*) assigning a unique role to every user, which makes $\mathcal{UA} = \{(u, r_u) \mid u \in \mathcal{U}\}$ and leaves $\mathcal{PA} = \mathcal{UPA}$ (substituting u with r_u in $\mathcal{UPA}$); and *ii*) assigning a distinct role to each permission, resulting in $\mathcal{UA} = \mathcal{UPA}$ (substituting p with r_p in $\mathcal{UPA}$) while $\mathcal{PA} = \{(r_p, p) \mid p \in \mathcal{P}\}$.

Extended RBAC Components While the standard role mining problem focuses on finding a complete candidate role-set, one could also consider incomplete role-sets where some permissions remain uncovered by roles. These uncovered permissions can be handled via a *direct user-permission* assignment relation, $\mathcal{DUPA} \subseteq \mathcal{U} \times \mathcal{P}$. Although not part of standard Core RBAC, this approach accommodates anomalous situations where a user's single permission assignment cannot be naturally generalized into a role. Furthermore, the NIST RBAC Reference Model extends beyond Core RBAC to include components such as Hierarchical RBAC (RBAC1). Hierarchical RBAC introduces a role hierarchy relation $\mathcal{RH} \subseteq \mathcal{R} \times \mathcal{R}$, denoted by $\succeq$. We write $r_1 \succeq r_2$ (role r_1 *inherits* role r_2) if all permissions assigned to r_2 are also assigned to r_1, and all users assigned to r_1 are also assigned to r_2. Finally, Constrained RBAC (RBAC2) is a NIST RBAC model component [19] that adds constraints to the core RBAC0 model to enforce organizational policies. Key features include mutual exclusion (preventing conflicting role assignments), role cardinality (limiting users per role), and prerequisite roles, which together minimize insider threats and ensure separation of duties. In this work, unless otherwise stated, we focus on Core RBAC without hierarchies, and we consider explicit families of constraints imposed on the role-to-permission assignments.

2.1 Static Separation of Permissions

This section explores an alternative method for introducing constraints into the role mining process. Specifically, we examine scenarios involving conflicting permissions, where certain predetermined groups of permissions are prohibited from being allocated to a single role. We designate this condition as *Static Separation of Permissions* (abbreviated as SSP). Conceptually, this aligns closely with *Static Separation of Duty Relations*, a model used to represent conflicts of interest within role-based systems [9]. Under the *Static Separation of Permissions* framework, it is permissible for a user to hold conflicting permissions, provided these permissions are associated with distinct roles. Consequently, to utilize conflicting permissions for accessing a *resource*, the user is required to

use separate roles. To facilitate further discussion, we define the following:

$$\mathsf{AssignedPrms_R}(r) = \{p : (r, p) \in \mathcal{PA}\} \text{ and}$$
$$\mathsf{AssignedPrms_U}(u) = \{p : (u, p) \in \mathcal{UPA}\}.$$

To formally represent the *Static Separation of Permissions* condition for a given configuration $\rho = \langle \mathcal{U}, \mathcal{P}, \mathcal{UPA} \rangle$, we utilize a collection of conflicting permission sets denoted as $\mathcal{CP} = \{C_1, \ldots, C_s\}$, where each $C_i \subseteq \mathcal{P} = \{p_1, \ldots, p_m\}$ for $1 \leq i \leq s$. An RBAC state $\gamma = \langle \mathcal{R}, \mathcal{UA}, \mathcal{PA} \rangle$ that is consistent with ρ is said to *satisfy* $\mathcal{CP}$ if the following holds for every role $r \in \mathcal{R}$ and every constraint $C \in \mathcal{CP}$,

$$C \not\subseteq \mathsf{AssignedPrms_R}(r). \tag{2}$$

Broadening this definition, a specific role r *satisfies* $\mathcal{CP}$ as long as Expression (2) remains true for all constraints $C \in \mathcal{CP}$. In contrast, a role $r \in \mathcal{R}$ is considered to *violate* a constraint $C \in \mathcal{CP}$ if $C \subseteq \mathsf{AssignedPrms_R}(r)$. To maintain uniform terminology, we refer to $\mathcal{CP}$ as a family of constraints and individual sets of conflicting permissions $C \in \mathcal{CP}$ simply as constraints.

Based on Expression (2), if a constraint $C \in \mathcal{CP}$ is satisfied by a role $r \in \mathcal{R}$, any larger constraint $C' \in \mathcal{CP}$ where $C \subset C'$ is also automatically satisfied by that role. Consequently, we operate under the assumption that there are no redundant constraints within the family $\mathcal{CP}$; meaning that for any two distinct constraints $C_i, C_j \in \mathcal{CP}$, the relations $C_i \subset C_j$ and $C_j \subset C_i$ are both false. Furthermore, single-element sets serve no logical purpose in $\mathcal{CP}$. If a singleton constraint $\{p\} \in \mathcal{CP}$ exists and a user $u \in \mathcal{U}$ has an assignment $(u, p) \in \mathcal{UPA}$, no valid RBAC state γ could possibly satisfy $\mathcal{CP}$. Indeed, Expression (2) would inevitably be violated by the role that contains p and is assigned to u throught (u, p). Therefore, it is assumed that $|C| > 1$ for all constraints $C \in \mathcal{CP}$. By combining these observations, we arrive at the definition of a *well-defined* family of constraints.

Definition 1. *The family conflicting permission sets $\mathcal{CP} = \{C_1, \ldots, C_s\}$ is considered a* well defined *family of constraints if every constraint $C \in \mathcal{CP}$ has a size $|C| > 1$, and for all pairs of distinct sets $C_i, C_j \in \mathcal{CP}$, neither $C_i \subset C_j$ nor $C_j \subset C_i$ holds.*

When provided with an RBAC configuration $\rho = \langle \mathcal{U}, \mathcal{P}, \mathcal{UPA} \rangle$ and a constraint family $\mathcal{CP}$, the objective is to locate an RBAC state $\gamma = \langle \mathcal{R}, \mathcal{UA}, \mathcal{PA} \rangle$ that both aligns with ρ and satisfies $\mathcal{CP}$. We term this process of deriving γ consistent with ρ and compliant with $\mathcal{CP}$ as the *role mining under static separation of permissions* problem (or the SSP problem, for short). It is important to note that a valid solution for SSP is invariably present. For any given RBAC configuration $\rho = \langle \mathcal{U}, \mathcal{P}, \mathcal{UPA} \rangle$ and constraint family $\mathcal{CP}$, a compliant and consistent state $\gamma = \langle \mathcal{R}, \mathcal{UA}, \mathcal{PA} \rangle$ can be generated by establishing a unique role for every permission $p \in \mathcal{P}$, and subsequently assigning this role to all users

$u \in \mathcal{U}$ where $(u, p) \in \mathcal{UPA}$. This trivial solution is expressed as

$$\begin{aligned} \mathcal{R} &= \{r_p : p \in \mathcal{P}\}, \\ \mathcal{UA} &= \{(u, r_p) : p \in \mathsf{AssignedPrms}_{\mathsf{U}}(u)\}, \text{ and} \\ \mathcal{PA} &= \{(r_p, p) : p \in \mathcal{P}\}. \end{aligned}$$

Should the objective shift towards identifying a role set $\mathcal{R}$ with the smallest possible size, we formulate the Static Separation of Permissions Role Mining Optimization problem (abbreviated as SSP-RM Optimization) as detailed below.

Problem 2. (SSP-RM Optimization) *For a provided RBAC configuration $\rho = \langle \mathcal{U}, \mathcal{P}, \mathcal{UPA} \rangle$ and a constraint family $\mathcal{CP}$, what is the smallest integer k that permits an RBAC state $\gamma = \langle \mathcal{R}, \mathcal{UA}, \mathcal{PA} \rangle$ satisfying $\mathcal{CP}$ with a role count $|\mathcal{R}| = k$?*

The corresponding decision variant of Problem 2, named SSP-RM, is defined as follows.

Problem 3. (SSP-RM) *Given an RBAC configuration $\rho = \langle \mathcal{U}, \mathcal{P}, \mathcal{UPA} \rangle$, a family of constraints $\mathcal{CP}$, and an integer parameter k, does there exist a Core RBAC state $\gamma = \langle \mathcal{R}, \mathcal{UA}, \mathcal{PA} \rangle$ that satisfies $\mathcal{CP}$ with $|\mathcal{R}| \leq k$?*

One can prove the NP-completeness of the SSP-RM problem by first demonstrating its membership in NP, and subsequently providing a polynomial-time reduction from the Constrained Set Basis problem. Before defining this problem below, we need to introduce some notation. The *support* of a family of sets $\mathcal{B}$ is the union of the sets in $\mathcal{B}$, that is

$$\mathsf{support}[\mathcal{B}] = \bigcup_{B \in \mathcal{B}} B.$$

Problem 4. (Constrained Set Basis) *For a collection $\mathcal{F}$ of subsets originating from a finite set $\mathcal{S}$, and a positive integer $k \leq |\mathcal{S}|$, does a subset family $\mathcal{B}$ of $\mathcal{S}$ exist such that $|\mathcal{B}| \leq k$, $\mathcal{B} \cap \mathcal{F} = \emptyset$, and for every $F \in \mathcal{F}$, there is a corresponding sub-family $\mathcal{B}_F \subseteq \mathcal{B}$ whose support is exactly F (i.e., $\mathtt{support}[\mathcal{B}_F] = F$)?*

Here, the family $\mathcal{B}$ acts as the *basis* for $\mathcal{F}$, meaning every set within $\mathcal{F}$ can be seen as a union of some sets in $\mathcal{B}$. Problem 4 represents a minor modification of the decisional version of Set Basis (see Problem SP7 in Garey and Johnson's book [11]) which was shown to be NP-complete by Stockmeyer [21]. The novel constraint applied to Set Basis dictates that the derived basis $\mathcal{B}$, comprised of $\mathcal{S}$'s subsets, is forbidden from encompassing any set present in $\mathcal{F}$. A formal proof that the Constrained Set Basis problem is NP-Complete can be built upon the established NP-completeness proof for the standard Set Basis problem that can be found in [21]. In brief, the proof shows that the Vertex Cover problem can be reduced in polynomial time to the Constrained Set Basis problem.

Theorem 1. SSP-RM *is NP-complete.*

Proof. The SSP-RM problem belongs to NP. This is because a proposed role set $\mathcal{R}$, paired with the `UA` and `PA` matrices, functions as a valid certificate/witness that allows for verification in polynomial time. To demonstrate hardness, examine an instance of the Constrained Set Basis problem, consisting of a subset family $\mathcal{F}$ of a finite set $\mathcal{S}$, and a positive integer $k \leq |\mathcal{S}|$. We construct an equivalent instance of the SSP-RM problem as follows. For each distinct element $f \in \texttt{support}[\mathcal{F}] \subseteq \mathcal{S}$, a corresponding permission $p_f \in \mathcal{P}$ is established. Concurrently, a user $u_F \in \mathcal{U}$ is introduced for each subset $F \in \mathcal{F}$. Furthermore, for any pairing (F, f) where $f \in F$, the user-permission assignment (u_F, p_f) is included in $\mathcal{UPA}$. Lastly, a constraint $C \in \mathcal{CP}$ is naturally induced by each set $F \in \mathcal{F}$. This entire reduction is clearly computable in polynomial time, and any valid solution $\gamma = \langle \mathcal{R}, \mathcal{UA}, \mathcal{PA} \rangle$ derived for the newly formed SSP-RM instance yields an immediate solution to the original Constrained Set Basis instance (the solution maps as $\mathcal{B} = \mathcal{R}$ and $\mathcal{B}_F = \{\mathsf{AssignedPrms}_{\mathsf{R}}(r) : (u_F, r) \in \mathcal{UA}\}$). Therefore, the theorem follows. □

Generally, computing an optimal solution for the SSP-RM problem is hard. The proof for the next obvious result is omitted.

Theorem 2. SSP-RM Optimization *is NP-hard.*

While Constrained Set Basis is established as NP-complete, it is worth noting that when the subset family $\mathcal{F}$ of the finite set $\mathcal{S}$ demonstrates specific structural properties, the problem becomes efficiently solvable. First, if $\mathcal{F}$ includes any singleton set, say X, no constrained set basis $\mathcal{B}$ can exist: covering X requires $X \in \mathcal{B}$, which contradicts the very definition of a *basis*. Conversely, if every set $F \in \mathcal{F}$ has a size of two, then a valid basis is simply $\mathcal{B} = \{\{x\} : x \in \texttt{support}[\mathcal{F}]\}$.

3 Post-processing Heuristic

In this section, we introduce a heuristic designed to operate within a post-processing framework [12]. Specifically, our algorithm takes an initial set of roles and user assignments as input and systematically modifies them to satisfy predefined constraints. In essence, post-processing role mining constitutes a two-stage approach to RBAC configuration. First, a standard, unconstrained role mining algorithm [8, 22, 18, 23] is executed to generate a baseline set of raw roles and user assignments. Following this, a refinement phase analyzes the generated roles to identify any violations of security or administrative constraints. If necessary, this phase alters, eliminates, or creates new roles to ensure strict constraint satisfaction [15, 2, 16].

This post-processing methodology generally offers greater flexibility compared to concurrent processing techniques [13, 12]. In concurrent processing, constraints are strictly enforced at every computational step, fundamentally dictating how new roles and user-permission assignments are initially constructed.

By decoupling the mining and refinement stages, the post-processing paradigm accommodates the use of any off-the-shelf mining algorithm. This makes it particularly advantageous for refining existing, complex, or noisy datasets. Furthermore, post-processing heuristics are frequently employed in scenarios requiring the strict enforcement of business rules or separation of duty (SoD) policies [9], such as capping the number of users assigned to a privileged role [7], after the initial mining phase to guarantee compliance.

In the following heuristic, to simplify the notation, we adopt the following conventions. For the role-to-permission assignment relation $\mathcal{PA}$, writing $r \in \mathcal{PA}$ means that the role r belongs to the set $\{r \mid (r,p) \in \mathcal{PA}\}$, that is, r is selected in the relation. The same convention applies to the other assignment relations. Furthermore, the symbols r, r_1, and r_2 denote both the roles themselves and the corresponding sets of assigned permissions. Accordingly, the notation $\mathcal{PA} \cup \{r\}$ stands for $\mathcal{PA} \cup \{(r,p) \mid r \in p\}$ (and analogously for $\mathcal{PA} \setminus \{r\}$). Moreover, the notation $\mathsf{Roles}(u) = \{r : (u,r) \in \mathcal{UA}\}$ will also be used throughout the description of the heuristic.

The intuition underlying the post-processing heuristic is the following. The function `CHECKVIOLATIONS` (line 3), given as input the role-to-permission assignment relation $\mathcal{PA}$ and the family of constraints $\mathcal{CP}$, identifies all roles that violate $\mathcal{CP}$. More precisely, it returns a set of pairs $(r, \mathit{violated})$, where r is a violating role and $\mathit{violated}$ is the set of constraints $c \in \mathcal{CP}$ such that $c \subseteq r$. Then (lines 4–7), for each violating role r, the function `SPLIT` partitions the permissions assigned to r into two new roles, say r_1 and r_2, in such a way that both satisfy $\mathcal{CP}$. In particular, see lines 21–24, for a given role r, the function `SPLIT` processes each violated constraint, randomly selects one permission from it, and assigns it to a new role r_1. It then returns the pair r_1 and $r_2 = r \setminus r_1$. Subsequently, the function `UPDATE` modifies the decomposition $(\mathcal{UA}_{\mathsf{pp}}, \mathcal{PA}_{\mathsf{pp}})$, which is initially set equal to $(\mathcal{UA}, \mathcal{PA})$ at line 2, to reflect this change. In particular, `UPDATE` replaces role r with the newly created roles r_1 and r_2 in both $\mathcal{UA}_{\mathsf{pp}}$ and $\mathcal{PA}_{\mathsf{pp}}$. That is, the function `UPDATE` scans all users to identify those assigned to the original role r and replaces r with r_1 and r_2 in their assignments (lines 28–32). Finally, it updates the role-to-permission relation by substituting r with r_1 and r_2 (line 33).

Time Complexity. The function `CHECKVIOLATIONS` iterates over every role $r \in \mathcal{PA}$. For each role, it verifies whether there exists a constraint $c \in \mathcal{CP}$ such that $c \subseteq r$. The outer loop executes $k = |r \mid (r,p) \in \mathcal{PA}|$ times, while the inner loop examines all $nc = |\mathcal{CP}|$ constraints. Checking whether a constraint is a subset of a role requires $\mathcal{O}(c_{\max})$ time, where $c_{\max} = \max\{|c| : c \in \mathcal{CP}\}$. Consequently, the overall time complexity of `CHECKVIOLATIONS` is $\mathcal{O}(k \cdot nc \cdot c_{\max})$. In the function `SPLIT`, the for-loop (lines 21–24) runs $|\mathit{violated}|$ times. Selecting a random permission and inserting it into r_1 takes $\mathcal{O}(1)$ time. Computing the set difference $r \setminus r_1$ requires at most $\mathcal{O}(m)$ time, where $m = |\mathcal{P}|$. Therefore, the overall time complexity of `SPLIT` is $\mathcal{O}(|\mathit{violated}| + m)$. In the worst case, when a role violates all constraints, this is bounded by $\mathcal{O}(nc + m)$. Finally,

Heuristic 1 Post-processing SSP

Require: A decomposition $(\mathcal{UA}, \mathcal{PA})$ and a family of constraints $\mathcal{CP}$
Ensure: A decomposition $(\mathcal{UA}_{\mathsf{pp}}, \mathcal{PA}_{\mathsf{pp}})$ satisfying $\mathcal{CP}$

procedure PP-SSP$(\mathcal{UA}, \mathcal{PA}, \mathcal{CP})$
  $(\mathcal{UA}_{\mathsf{pp}}, \mathcal{PA}_{\mathsf{pp}}) = (\mathcal{UA}, \mathcal{PA})$
  $\mathcal{V} =$ CHECKVIOLATIONS$(\mathcal{PA}, \mathcal{CP})$ ▷ Identify roles violating $\mathcal{CP}$
  for all $(r, violated) \in \mathcal{V}$ **do**
    $r_1, r_2 =$ SPLIT$(r, violated)$ ▷ Decompose r into two roles
    UPDATE$(\mathcal{UA}_{\mathsf{pp}}, \mathcal{PA}_{\mathsf{pp}}, r, r_1, r_2)$ ▷ Substitute r with r_1 and r_2
  end for
  return $(\mathcal{UA}_{\mathsf{pp}}, \mathcal{PA}_{\mathsf{pp}})$
end procedure

function CHECKVIOLATIONS$(\mathcal{PA}, \mathcal{CP})$
  for all $r \in \mathcal{PA}$ **do**
    $violated = \{c \in \mathcal{CP} \mid c \subseteq r\}$
    if $violated\ != \emptyset$ **then** ▷ Role r violates $\mathcal{CP}$
      $\mathcal{V} = \mathcal{V} \cup \{(r, violated)\}$
    end if
  end for
  return $\mathcal{V}$
end function

function SPLIT$(r, violated)$ ▷ Decompose r into two roles
  $r_1 = \emptyset$
  for all $constraint \in violated$ **do**
    $p \leftarrow_R constraint$ ▷ Select a random permission p
    $r_1 = r_1 \cup \{p\}$
  end for
  return $r_1,\ r \backslash r_1$
end function

function UPDATE$(\mathcal{UA}_{\mathsf{pp}}, \mathcal{PA}_{\mathsf{pp}}, r, r_1, r_2)$
  for all $u \in \mathcal{UA}_{\mathsf{pp}}$ **do**
    if $r \in \mathsf{Roles}(u)$ **then** ▷ Substitute r with r_1 and r_2
      $\mathcal{UA}_{\mathsf{pp}} = \mathcal{UA}_{\mathsf{pp}} \backslash \{(u, r)\} \cup \{(u, r_1), (u, r_2)\}$
    end if
  end for
  $\mathcal{PA}_{\mathsf{pp}} = \mathcal{PA}_{\mathsf{pp}} \backslash \{r\} \cup \{r_1, r_2\}$ ▷ Delete r from $\mathcal{PA}_{\mathsf{pp}}$, add r_1 and r_2
end function

in the function UPDATE, the for-loop iterates over all $n = |\mathcal{U}|$ users. Checking whether $r \in \mathsf{Roles}(u)$ and updating the corresponding assignments takes $\mathcal{O}(1)$ time when sets are implemented using hash tables. Updating $\mathcal{PA}_{\mathsf{pp}}$ also requires $\mathcal{O}(1)$ time. Hence, the overall time complexity of UPDATE is $\mathcal{O}(n)$.

In summary, the heuristic PP-SSP first creates a copy of the assignment relations (line 2), which requires time $\mathcal{O}(n{\cdot}k{+}k{\cdot}m)$. It then invokes CHECKVIOLATIONS to detect all constraint violations and iterates over each violating role ($r \in \mathcal{V}$). For every such role, the functions SPLIT and UPDATE are executed to resolve the violation by partitioning the permissions assigned to r and updating the assignment relations accordingly. In the worst case, a constraint may involve all m permissions, and the set $\mathcal{V}$ may include all k roles. Therefore, the overall time complexity of PP-SSP is

$$\mathcal{O}(n \cdot k + k \cdot m + k \cdot nc \cdot m + k \cdot (nc + m + n)) = \mathcal{O}(k \cdot (n + m + nc \cdot m)).$$

Hence, the most computationally demanding phase of the algorithm is the detection of violations, whose cost grows linearly with the number of roles, the number of constraints, and the size of the largest constraint.

4 Experimentation Pipeline

In this section, we present the experimentation pipeline used to test the proposed post-processing role mining technique. The heuristic has been implemented in Python and is publicly available on GitHub [4]. All experiments were conducted on a MacBook Air running macOS 15.5, equipped with an Apple M4 processor and 24 GB of LPDDR5 RAM. For the evaluation, we used benchmark datasets derived from both real-world scenarios [8] and synthetically generated data [24]. We report a representative subset of the experiments performed on these datasets. The complete set of experimental results is available in the online supplementary material [4].

The experimentation pipeline used in the experiments is the following.

1. Consider the decomposition $(\mathcal{UA}, \mathcal{PA})$ and compute $\mathcal{UPA}$ from it.
2. Given $(\mathcal{UA}, \mathcal{PA}, \mathcal{UPA})$, generate the family of constraints $\mathcal{CP}$.
3. For each variant $\mathsf{V} \in \{\mathtt{OL}, \mathtt{OI}, \mathtt{UL}, \mathtt{UI}\}$,
   (a) Run RM-IDF on $\mathcal{UPA}$ and V, obtaining the decomposition $(\mathcal{UA}_{\mathtt{rm}}, \mathcal{PA}_{\mathtt{rm}})$.
   (b) Run PP-SSP on $(\mathcal{UA}_{\mathtt{rm}}, \mathcal{PA}_{\mathtt{rm}})$ and $\mathcal{CP}$, obtaining the decomposition $(\mathcal{UA}_{\mathtt{ssp}}, \mathcal{PA}_{\mathtt{ssp}})$.
   (c) Compute $\mathsf{Sim}_{\mathtt{d}}$ and $\mathsf{Jacc}_{\mathtt{d}}$, for $\mathtt{d} \in \{\mathtt{gm}, \mathtt{gp}, \mathtt{mp}\}$.

Figure 1: Experimentation Pipeline

The decomposition $(\mathcal{UA}, \mathcal{PA})$ considered in Step 1 of Figure 1 is either the optimal decomposition of real-world datasets reported in [8], or the synthetic decomposition generated according to the random data generator, first introduced in [24].

In Step 2 of Figure 1, the family of constraints $\mathcal{CP}$ is generated according to four modalities: `upa-rnd`, `upa-small`, `pa-rnd`, and `pa-small`. The modalities `upa-rnd` and `upa-small` select constraints from the user-to-permission relation $\mathcal{UPA}$, whereas `pa-rnd` and `pa-small` select them from the role-to-permission relation $\mathcal{PA}$. The `rnd` option selects the permissions associated with a constraint c_i, for $i = 1, \ldots, |\mathcal{CP}|$, by choosing a user u_i (respectively, a role r_i) uniformly at random and setting

$$c_i = \{p : (u_i, p) \in \mathcal{UPA}\} \quad \text{(respectively, } c_i = \{p : (r_i, p) \in \mathcal{PA}\}\text{)}.$$

The `small` option instead selects $|\mathcal{CP}|$ users (respectively, roles) having the smallest permission sets (in terms of cardinality). The corresponding constraints are then constructed in the same way as in the `rnd` option. For both options, when adding constraints to $\mathcal{CP}$, we ensure that the resulting family is well defined; that is, no constraint is contained in, nor contains, another constraint in $\mathcal{CP}$.

The heuristic `RM-IDF`, used in the Step 3.(a) of Figure 1, is the role mining heuristic described in [3] applied to the unconstrained scenario. That is, the constraint on the size of each role has been dropped, we run the heuristic assuming the maximum number of permissions included in each role can be as large as $|\mathcal{P}|$. The heuristic `RM-IDF` was derived from the `RM` algorithm introduced in [6], with several adaptations to better suit the constrained context. It adopts two strategies for selecting the permissions used to construct a role. Permissions are drawn either from a row of the original user–permission assignment matrix `UPA` (strategy `O`), or from a row of the matrix of uncovered permissions, permissions that are not yet fully represented by the current set of roles, denoted by `uncUPA` (stategy `U`). The first strategy, based on `UPA`, exploits the complete user-permission information to guide the role construction process. In contrast, the second strategy relies on `uncUPA`, thereby operating on a progressively smaller instance, since the number of uncovered permissions decreases at each step of the mining procedure. Given a user–permission assignment matrix (either `UPA` or `uncUPA`), the heuristic selects users according to one of two criteria: the number of assigned permissions (i.e., the *cardinality* of their permission set) or the total *IDF* score (i.e., the sum of the *IDF* values[1] of their assigned permissions). Under the first criterion, the user with the smallest number of permissions is selected (strategy `L`). Under the second, the user with the lowest total *IDF* score is chosen (strategy `I`). The choice of the user-permission assignment matrix, together with the user selection criterion, determines the four possible variants `OL`, `OI`, `UL`, and `UI` used in Step 3 of Figure 1. These variants specify the strategy adopted to generate new candidate roles.

[1]The *IDF* value for the permission p, assigned to at least one user, is defined as the logarithm of the ratio between the number of users $|\mathcal{U}|$ and the number of users to whom permission p is assigned $|\{u \in \mathcal{U} : (u, p) \in \mathcal{UPA}\}|$.

To evaluate the *quality* of the decompotition computed by the heuristic `PP-SSP`, we will use: the role-set size $|\mathcal{R}|$, the *Weighted Structural Complexity* `WSC`, the *Jaccard Index* `Jacc`, and the *Similarity* `Sim`.

The Weighted Structural Complexity `WSC` is a metric introduced in [14, 17] to evaluate the size of RBAC systems, measuring the total cost of roles, user-role assignments, and role-permission assignments. It weights different components to balance security and administration, aiming for fewer roles and simpler structures. The `WSC` it is defined as follows.

Definition 2. *Given* $W = \langle w_r, w_u, w_p, w_h, w_d \rangle$, *where* $w_r, w_u, w_p, w_h, w_d \in \mathbb{Q}^+ \cup \{\infty\}$, *the* Weighted Structural Complexity (`WSC`) *of an RBAC state* $\gamma = \langle \mathcal{R}, \mathcal{UA}, \mathcal{PA} \rangle$, *denoted by* $wsc(\gamma, W)$, *is computed as follows.*

$$wsc(\gamma, W) = w_r \cdot |\mathcal{R}| + w_u \cdot |\mathcal{UA}| + w_p \cdot |\mathcal{PA}| + w_h \cdot |t_{reduce}(\mathcal{RH})| + w_d \cdot |\mathcal{DUPA}|, \tag{3}$$

where $t_{reduce}(\mathcal{RH})$ *is the transitive reduction of the role hierarchy* $\mathcal{RH} \subseteq \mathcal{R} \times \mathcal{R}$ *(a.k.a,* inheritance *relation) and* $\mathcal{DUPA} \subseteq \mathcal{U} \times \mathcal{P}$ *is the* direct user-permission *assignment relation, while* $|\cdot|$ *denotes the size of the relation.*

Given two set of roles, say $\mathcal{R}_1$ and $\mathcal{R}_2$, the *Jaccard Index* (also known as *Jaccard Similarity*), denoted by $\mathtt{Jacc}(\mathcal{R}_1, \mathcal{R}_2)$, measures their similarity. It is defined as the ratio of their intersection and the size of their union, namely $\mathsf{Jacc}(\mathcal{R}_1, \mathcal{R}_2) = |\mathcal{R}_1 \cap \mathcal{R}_2| / |\mathcal{R}_1 \cup \mathcal{R}_2|$. This measure can naturally be extended to a pair of roles. In this case, given two roles r_i and r_j (with a slight abuse of notation, we use r to denote both the role and the set of its assigned permissions), their Jaccard Index (i.e., their role-to-role similarity) is defined as $Jaccard(r_i, r_j) = |r_i \cap r_j| / |r_i \cup r_j|$. The *Similarity* metric Sim, based on the role-to-role Jaccard Index, evaluates how closely a role set $\mathcal{R}_1$resembles another role set $\mathcal{R}_2$. This measure was originally introduced in [18] and subsequently adopted in [10]. The similarity between $\mathcal{R}_1$ and $\mathcal{R}_2$ is computed by, for each role in $\mathcal{R}_1$, determining the maximum Jaccard Index with any role in $\mathcal{R}_2$, and then averaging these maximum values. Formally,

$$\overline{\mathsf{Sim}}(\mathcal{R}_1, \mathcal{R}_2) = \frac{\sum_{r_i \in \mathcal{R}_1} \max_{r_j \in \mathcal{R}_2} Jaccard(r_i, r_j)}{|\mathcal{R}_1|}.$$

Following the approach proposed in [1], we compute the final similarity as the average of the measure evaluated in both directions.

$$\mathsf{Sim}(\mathcal{R}_1, \mathcal{R}_2) = \frac{\overline{\mathsf{Sim}}(\mathcal{R}_1, \mathcal{R}_2) + \overline{\mathsf{Sim}}(\mathcal{R}_2, \mathcal{R}_1)}{2}.$$

The resulting similarity score ranges from 0 to 1, where a value of 1 indicates that the role set $\mathcal{R}_1$ exactly matches the role set $\mathcal{R}_2$.

Denoting with $\mathcal{R}$, $\mathcal{R}_{\mathtt{rm}}$, and $\mathcal{R}_{\mathtt{ssp}}$ the role sets induced by $\mathcal{PA}$, $\mathcal{PA}_{\mathtt{rm}}$, and $\mathcal{PA}_{\mathtt{ssp}}$, respectively, the measures $\mathsf{Sim}_{\mathtt{d}}$ and $\mathsf{Jacc}_{\mathtt{d}}$, for $\mathtt{d} \in \{\mathtt{gm}, \mathtt{gp}, \mathtt{mp}\}$, computed

at Step 3.(c) of Figure 1, are defined as:

$$\mathsf{Sim}_{\mathsf{gm}} = \mathsf{Sim}(\mathcal{R}, \mathcal{R}_{\mathtt{rm}}) \qquad \mathsf{Jacc}_{\mathsf{gm}} = \mathsf{Jacc}(\mathcal{R}, \mathcal{R}_{\mathtt{rm}})$$
$$\mathsf{Sim}_{\mathsf{gp}} = \mathsf{Sim}(\mathcal{R}, \mathcal{R}_{\mathtt{ssp}}) \qquad \mathsf{Jacc}_{\mathsf{gp}} = \mathsf{Jacc}(\mathcal{R}, \mathcal{R}_{\mathtt{ssp}})$$
$$\mathsf{Sim}_{\mathsf{mp}} = \mathsf{Sim}(\mathcal{R}_{\mathtt{rm}}, \mathcal{R}_{\mathtt{ssp}}) \qquad \mathsf{Jacc}_{\mathsf{mp}} = \mathsf{Jacc}(\mathcal{R}_{\mathtt{rm}}, \mathcal{R}_{\mathtt{ssp}}).$$

4.1 Experiments on Real-World Datasets

A collection of nine real-world datasets has been widely used to evaluate the performance of role mining algorithms. In this section, we present the results of our experiments conducted on these real-world datasets, which were first introduced for role mining evaluation in [8]. The datasets, listed in Table 1, were obtained from HP Labs and originally employed in [8] for benchmarking purposes. These datasets vary significantly in size and density, ranging from very sparse (e.g., *Apj*) to very dense (e.g., *Healthcare*). For eight of these datasets, the optimal solutions to the unconstrained role mining problem are known and were reported in [8]. Table 1 summarizes the main characteristics and key parameters of the datasets.

Dataset	$\lvert\mathcal{U}\rvert$	$\lvert\mathcal{P}\rvert$	$\lvert\mathcal{UPA}\rvert$	$\lvert\mathcal{R}\rvert$	Density
Americas large	3485	10127	185294	398	0.00525
Americas small	3477	1587	105205	178	0.01907
Apj	2044	1164	6841	453	0.00288
Customer	10021	277	45427	//	0.01637
Domino	79	231	730	20	0.04000
Emea	35	3046	7220	34	0.06772
Firewall 1	365	709	31951	66	0.12347
Firewall 2	325	590	36428	10	0.18998
Healthcare	46	46	1486	14	0.70227

Table 1: Real world datasets

Based on the experimentation pipeline described in Section 4, the Tables 2–5 report the performance of the post-processing static separation of permissions heuristic `PP-SSP` across eight of the real-world datasets summarized in Table 1. The experimentation pipeline evaluates the heuristic using four different modalities for generating the constraint family $\mathcal{CP}$, the four tables correspond to the four constraint-generation modalities: `upa-rnd`, `upa-small`, `pa-rnd`, and `pa-small`, respectively. In all experiments, we set $|\mathcal{CP}| = 8$. For all datasets except *Customer*, these tables report the minimum number of roles, $|\mathcal{R}|$, achieved after post-processing the unconstrained decomposition $(\mathcal{UA}_{\mathtt{rm}}, \mathcal{PA}_{\mathtt{rm}})$. This initial decomposition is generated by executing the `RM-IDF` algorithm using a variant $\mathsf{V} \in \{\mathtt{OL}, \mathtt{OI}, \mathtt{UL}, \mathtt{UI}\}$; the specific variant that yields this minimal role count is denoted in the second column. Furthermore, the tables detail the resulting Weighted Structural Complexity (WSC) alongside two sets of similarity metrics:

$\mathsf{Sim}_{\mathsf{gp}}$ and $\mathsf{Jacc}_{\mathsf{gp}}$ (which compare the original decomposition against the post-processed one), and $\mathsf{Sim}_{\mathsf{mp}}$ and $\mathsf{Jacc}_{\mathsf{mp}}$ (which compare the intermediate mined roles against the post-processed state).

Across almost all datasets and constraint generation modalities, the `OI` variant clearly emerges as the dominant one. This observation suggests that combining the original UPA matrix (`O`) with IDF-based user selection (`I`) provides a consistently robust strategy. In particular, the IDF criterion appears effective in generating role sets that remain structurally stable even after the enforcement of constraints. Empirically, the prevalence of the `OI` variant indicates that prioritizing users with lower aggregate IDF scores tends to produce roles that require only limited restructuring during the repair phase.

The number of roles $|\mathcal{R}|$ generally experiences a slight increase after post-processing. For the majority of datasets, this growth is moderate (e.g., the Americas large dataset shifts from 419 to 421 roles). While some datasets, such as Healthcare under the `pa-small` modality, exhibit a more significant expansion, the overall increase remains bounded. This implies that constraint violations tend to be localized rather than pervasive throughout the system. However, dense datasets like Healthcare demonstrate heightened sensitivity to the chosen constraint selection strategy, suffering more severe impacts under specific modalities.

Dataset	V	$\lvert\mathcal{R}\rvert$	WSC	$\mathsf{Sim}_{\mathsf{gp}}$	$\mathsf{Jacc}_{\mathsf{gp}}$	$\mathsf{Sim}_{\mathsf{mp}}$	$\mathsf{Jacc}_{\mathsf{mp}}$
Americas large	`OI`	419	95994	0.9095	0.3801	0.9927	0.9576
Americas small	`OL`	197	14058	0.7468	0.2295	0.9973	0.9848
Apj	`OI`	458	5242	0.8826	0.5363	0.9936	0.974
Domino	`OI`	20	761	0.9017	0.3333	1.0	1.0
Emea	`OL`	42	7296	0.9048	0.52	0.9048	0.52
Firewall 1	`OI`	67	3297	0.7793	0.3039	0.9803	0.913
Firewall 2	`OI`	11	1850	0.7882	0.1667	0.9516	0.75
Healthcare	`UI`	14	351	0.7469	0.3333	1.0	1.0

Table 2: Real-world datasets – $\mathcal{CP}$ generation modality `upa-rnd`

The values of the Weighted Structural Complexity (WSC) largely follow the trend observed for the number of roles, although they are also affected by changes in the user–role $\mathcal{UA}$ and role–permission $\mathcal{PA}$ relations. For larger datasets such as Americas large, the WSC remains within the same order of magnitude across all modalities. Conversely, smaller datasets, such as Healthcare and Firewall 2, display more pronounced WSC fluctuations. As anticipated, constraints derived directly from role-permission relations (the `pa-*` modalities) tend to interfere more heavily with the mined structure. Furthermore, the `small` modality generally induces greater structural distortion compared to the random (`rnd`) approach.

When evaluating the similarity between the intermediate mined roles and the final post-processed roles, the data reveals that $\mathsf{Sim}_{\mathsf{mp}}$ is consistently high

Dataset	V	$\lvert\mathcal{R}\rvert$	WSC	$\mathsf{Sim}_{\mathsf{gp}}$	$\mathsf{Jacc}_{\mathsf{gp}}$	$\mathsf{Sim}_{\mathsf{mp}}$	$\mathsf{Jacc}_{\mathsf{mp}}$
Americas large	OI	421	93126	0.9059	0.3742	0.9891	0.9441
Americas small	UL	218	11175	0.6994	0.1681	0.9817	0.9196
Apj	OI	459	5307	0.8815	0.5328	0.9918	0.9677
Domino	OI	20	761	0.9017	0.3333	1.0	1.0
Emea	UL	41	7295	0.9078	0.5306	0.9078	0.5306
Firewall 1	OI	71	3593	0.7096	0.2342	0.9286	0.7436
Firewall 2	OI	12	1940	0.7446	0.1	0.9054	0.5714
Healthcare	UI	15	397	0.7189	0.2609	0.965	0.8125

Table 3: Real-world datasets – $\mathcal{CP}$ generation modality `upa-small`

Dataset	V	$\lvert\mathcal{R}\rvert$	WSC	$\mathsf{Sim}_{\mathsf{gp}}$	$\mathsf{Jacc}_{\mathsf{gp}}$	$\mathsf{Sim}_{\mathsf{mp}}$	$\mathsf{Jacc}_{\mathsf{mp}}$
Americas large	OI	419	93084	0.9097	0.3801	0.9927	0.9576
Americas small	OL	202	11382	0.7392	0.2141	0.9856	0.9135
Apj	OI	458	5161	0.8839	0.5363	0.9943	0.974
Domino	OI	20	761	0.9017	0.3333	1.0	1.0
Emea	OL	42	7296	0.9044	0.52	0.9044	0.52
Firewall 1	OI	66	3526	0.7895	0.32	0.9923	0.9552
Firewall 2	OI	11	1850	0.7882	0.1667	0.9516	0.75
Healthcare	UL	16	359	0.6735	0.2	0.8922	0.5789

Table 4: Real-world datasets – $\mathcal{CP}$ generation modality `pa-rnd`

Dataset	V	$\lvert\mathcal{R}\rvert$	WSC	$\mathsf{Sim}_{\mathsf{gp}}$	$\mathsf{Jacc}_{\mathsf{gp}}$	$\mathsf{Sim}_{\mathsf{mp}}$	$\mathsf{Jacc}_{\mathsf{mp}}$
Americas large	OI	420	93106	0.9054	0.3748	0.9877	0.9327
Americas small	OL	203	11397	0.7278	0.2019	0.9738	0.9
Apj	OI	459	5307	0.8815	0.5328	0.9918	0.9677
Domino	OI	20	761	0.9017	0.3333	1.0	1.0
Emea	OI	41	7295	0.9094	0.5306	0.9094	0.5306
Firewall 1	OI	71	3564	0.7084	0.2342	0.9273	0.7436
Firewall 2	OI	14	2094	0.6489	0.0	0.7959	0.3333
Healthcare	UL	20	446	0.5526	0.0303	0.7503	0.2593

Table 5: Real-world datasets – $\mathcal{CP}$ generation modality `pa-small`

across all tables, frequently exceeding 0.95. The corresponding Jaccard index, $\mathsf{Jacc}_{\mathsf{mp}}$, remains similarly elevated, except under the most aggressive constraint scenarios (e.g., Healthcare subjected to `pa-small`). These high fidelity metrics demonstrate that the `PP-SSP` heuristic is minimally invasive relative to the unconstrained mined decomposition. The necessary modifications manifest primarily as localized role splits rather than global systemic restructuring, proving that `PP-SSP` successfully resolves constraint violations while largely preserving

the underlying semantics and structure of the mining output.

However, a comparison between the original optimal roles and the post-processed roles reveals a more nuanced reality. The similarity to the original decomposition ($\mathsf{Sim}_{\mathsf{gp}}$) is notably lower, with $\mathsf{Jacc}_{\mathsf{gp}}$ values frequently falling between 0.2 and 0.5. Performance degrades particularly sharply under the `pa-small` modality, as evidenced by the Healthcare dataset, where $\mathsf{Jacc}_{\mathsf{gp}} \approx 0.03$. This divergence underscores that stringent constraint enforcement can drive the final decomposition significantly far from its original structure, highlighting the major influence of the constraint-generation strategy.

Finally, Table 5 reports an interesting case for the *Firewall 2* dataset under the `pa-small` modality, where $\mathsf{Jacc}_{\mathsf{gp}} = 0.0$, indicating absolutely zero intersection between the original role sets and the post-processed sets. Rather than being anomalous, this outcome can be explained by the experimental setup. In our experimental setup, the number of constraints is fixed at 8. The `pa-small` modality specifically selects the 8 *smallest* roles to serve as these constraints. Given that the optimal decomposition for Firewall 2 comprises exactly 10 roles (see Table 1), satisfying the constraint family $\mathcal{CP}$ dictates that the majority of the original roles must be fundamentally altered or split. Consequently, it is highly probable that the resulting post-processed roles will be entirely disjoint from the original set.

In conclusion, the results reported in Tables 2–5 indicate that the `PP-SSP` heuristic behaves in a structurally stable and minimally invasive manner with respect to the mined role sets. The `OI` variant consistently emerges as the most effective among the considered RM-IDF strategies. At the same time, the constraint-generation strategy has a noticeable impact on the degree of structural modification introduced during post-processing. In particular, the `pa-small` modality tends to produce the most significant alterations to the mined decomposition, while dense datasets appear to be more sensitive to the enforcement of separation constraints.

4.2 Experiments on Sythetic Datasets

To rigorously evaluate our heuristic, we also utilized synthetic datasets. We adopted the random data generator originally introduced in [24], which employs five key parameters to simulate diverse access control configurations: the total number of users nu, the total number of roles nr, the total number of permissions np, the maximum number of roles permissible for a single user mru, and the maximum number of permissions assignable to a single role mpr. An example of the parameters used in our experiments is given in Table 6.

The synthetic data generation process proceeds through three primary phases:

- *Role-Permission Assignment*: For every role, a random quantity of permissions (bounded by mpr) is uniformly selected and assigned.
- *User-Role Assignment*: Similarly, each user is allocated a random number of roles (bounded by mru).

- *User-Permission Computation*: Finally, the user-to-permission assignments are deterministically calculated by taking the boolean product of the generated user-role and role-permission assignment relations.

It is important to note that the datasets produced by this generator are inherently unstructured. The generation mechanism treats every user, role, and permission as a statistically independent entity, without embedding any underlying semantic or organizational hierarchy.

Dataset	nr	nu	np	$mrcu$	mpr
`d11`	100	2000	100	3	10
`d12`	100	2000	500	3	50
`d13`	100	2000	1000	3	100
`d14`	100	2000	2000	3	200

Table 6: Constant nr and nu, varying np, and $mrcu \times mpr \approx np/3.33$

.

The full set of experimental results can be found in the online supplementary material [4].

Acknowledgements

This work was supported in part by project SERICS (PE00000014) under the NRRP MUR program funded by the EU–NGEU. However, the views and opinions expressed are those of the authors alone and do not necessarily reflect those of the European Union or the Italian MUR. Neither the European Union nor the Italian MUR can be held responsible for them.

References

[1] Marco Benedetti and Marco Mori. Parametric RBAC maintenance via max-sat. In *SACMAT 2018*, pages 15–25, Indianapolis, IN, USA, June 13-15, 2018. ACM, New York.

[2] Carlo Blundo and Stelvio Cimato. Constrained role mining. In *STM 2012*, volume 7783 of *LNCS*, pages 289–304. Springer, 2013.

[3] Carlo Blundo and Stelvio Cimato. A bag of words model for efficient discovery of roles in access control systems. *Computers & Security*, 162:104808, 2026.

[4] Carlo Blundo and Stelvio Cimato. Static Separation of Permissions Heuristics. https://github.com/carblu/ssp, March 2026. GPL-3.0 license.

[5] Carlo Blundo, Stelvio Cimato, and Luisa Siniscalchi. PRUCC-RM: permission-role-usage cardinality constrained role mining. In *COMPSAC 2017. Volume 2*, pages 149–154, 2017.

[6] Carlo Blundo, Stelvio Cimato, and Luisa Siniscalchi. Managing constraints in role based access control. *IEEE Access*, 8:140497–140511, 2020.

[7] Carlo Blundo, Stelvio Cimato, and Luisa Siniscalchi. Heuristics for constrained role mining in the post-processing framework. *Journal of Ambient Intelligence and Humanized Computing*, 14(8):9925–9937, 2023.

[8] Alina Ene, William G. Horne, Nikola Milosavljevic, Prasad Rao, Robert Schreiber, and Robert Endre Tarjan. Fast exact and heuristic methods for role minimization problems. In Indrakshi Ray and Ninghui Li, editors, *SACMAT 2008*, pages 1–10. ACM, 2008.

[9] David F. Ferraiolo, Ravi S. Sandhu, Serban I. Gavrila, D. Richard Kuhn, and Ramaswamy Chandramouli. Proposed NIST standard for role-based access control. *ACM Transaction on Information System Security*, 4(3):224–274, 2001.

[10] Mario Frank, Joachim M. Buhmann, and David A. Basin. On the definition of role mining. In *SACMAT 2010*, pages 35–44. ACM, 2010.

[11] Michael R. Garey and David S. Johnson. *Computers and Intractability, A Guide to the Theory of NP-Completeness*. W.H. Freeman and Company, New York, 1979.

[12] Pullamsetty Harika, Marreddy Nagajyothi, John C. John, Shamik Sural, Jaideep Vaidya, and Vijayalakshmi Atluri. Meeting cardinality constraints in role mining. *IEEE Trans. Dependable Sec. Comput.*, 12(1):71–84, 2015.

[13] Ravi Kumar, Shamik Sural, and Arobinda Gupta. Mining RBAC roles under cardinality constraint. In *ICISS 2010*, volume 6503 of *LNCS*, pages 171–185, 2010.

[14] Ninghui Li, Ian Molloy, Qihua Wang, Elisa Bertino, Seraphin Calo, and Jorge Lobo. Role mining for engineering and optimizing role based access control systems. Technical report, Purdue University, 11 2007.

[15] Ruixuan Li, Huaqing Li, Xiwu Gu, Yuhua Li, Wei Ye, and Xiaopu Ma. Role mining based on cardinality constraints. *Concurr. Comput. : Pract. Exper.*, 27(12):3126–3144, August 2015.

[16] Haibing Lu, Jaideep Vaidya, Vijayalakshmi Atluri, and Yuan Hong. Constraint-aware role mining via extended boolean matrix decomposition. *IEEE Trans. Dependable Secur. Comput.*, 9(5):655–669, 2012.

[17] Ian Molloy, Hong Chen, Tiancheng Li, Qihua Wang, Ninghui Li, Elisa Bertino, Seraphin B. Calo, and Jorge Lobo. Mining roles with semantic meanings. In Indrakshi Ray and Ninghui Li, editors, *SACMAT 2008*, pages 21–30. ACM, 2008.

[18] Ian Molloy, Ninghui Li, Tiancheng Li, Ziqing Mao, Qihua Wang, and Jorge Lobo. Evaluating role mining algorithms. In Barbara Carminati and James Joshi, editors, *SACMAT 2009*, pages 95–104. ACM, 2009.

[19] NIST. Role Based Access Control. https://csrc.nist.gov/projects/role-based-access-control/faqs, 2026. Accessed: 2026-03-02.

[20] Ravi Sandhu, David Ferraiolo, and Richard Kuhn. The nist model for role-based access control: Towards a unified standard. In *RBAC '00*, pages 47–63. ACM, 2000.

[21] L. J. Stockmeyer. The minimal set basis problem is NP-complete. Technical Report RC 5431, IBM Research, May 1975.

[22] Jaideep Vaidya, Vijayalakshmi Atluri, and Qi Guo. The role mining problem: finding a minimal descriptive set of roles. In Volkmar Lotz and Bhavani M. Thuraisingham, editors, *SACMAT 2007*, pages 175–184. ACM, 2007.

[23] Jaideep Vaidya, Vijayalakshmi Atluri, and Qi Guo. The role mining problem: A formal perspective. *ACM Trans. Inf. Syst. Secur.*, 13(3), 2010.

[24] Jaideep Vaidya, Vijayalakshmi Atluri, and Janice Warner. Roleminer: mining roles using subset enumeration. In *CCS '06*, pages 144–153. ACM, 2006.